



genpact

Academic Project Report

End-to-End Salient Object Detection Using a Custom CNN

Project #3: End-to-End ML/DL Project

Repository: <https://github.com/yllnuredini/sod-project>

Dataset: ECSSD

Framework: PyTorch

Demo: Gradio

Prepared as a 6-10 page academic report covering data pipeline, architecture, training, evaluation, experiments, visualizations, demo, and tools.

Executive Summary

This report documents an end-to-end Salient Object Detection (SOD) system implemented with a custom Convolutional Neural Network built from scratch in PyTorch. The system receives an RGB image and predicts a one-channel saliency mask that highlights the visually dominant object or region in the scene. Unlike transfer-learning approaches, this project does not rely on pre-trained weights; the network learns its visual features directly from the selected public dataset.

The project uses the ECSSD dataset, resizes image-mask pairs to 224 x 224 pixels, applies normalization and training-only augmentation, and splits the data into 70% training, 15% validation, and 15% testing. The codebase is organized into separate modules for data loading, model definition, training, evaluation, and an interactive Gradio demo. The model follows an encoder-decoder design with convolutional layers, ReLU activations, max pooling, transposed convolution upsampling, batch normalization, skip connections, dropout, and a sigmoid output layer.

The training process uses a custom loss function that combines Binary Cross-Entropy with an IoU-based loss term. The evaluation script calculates Intersection-over-Union, Precision, Recall, F1-Score, Mean Absolute Error, and inference time. The project also includes visualization logic for comparing input images, ground-truth masks, predicted masks, and overlay outputs.

1. Introduction

Salient Object Detection is a computer vision task focused on identifying the most visually important object or region in an image. Instead of classifying every object in a scene, SOD tries to separate the region that naturally attracts human attention from the surrounding background. The expected output is a saliency mask, where high pixel values represent salient foreground regions and low pixel values represent background.

The goal of this project is to design and implement a complete SOD system from scratch using foundational Machine Learning and Deep Learning concepts. The model is a custom Convolutional Neural Network, meaning the architecture is manually designed and trained without using a pre-trained backbone. This approach is important because it demonstrates understanding of the full pipeline rather than simply reusing an existing segmentation model.

The system is expected to take any input image, process it into the required model format, predict a saliency mask, and visualize the result as both a standalone mask and an overlay on the original image. The complete implementation covers data preparation, augmentation, model architecture, training logic, checkpoint saving, evaluation metrics, visualization, and a demonstration interface.

2. Data Pipeline

2.1 Selected Dataset

The selected public dataset is ECSSD. In the codebase, the dataset is stored under `data/ECSSD`, with input images in the `images` folder and ground-truth masks in the `ground_truth_mask` folder. This dataset is appropriate for SOD because it provides natural images together with pixel-level saliency annotations.

The custom dataset class loads each image as an RGB image and each mask as a grayscale image. This matches the model requirements: the input has three color channels, while the output target is a single-channel saliency mask.

2.2 Resizing and Normalization

All input images and masks are resized to 224 x 224 pixels. This produces a consistent input shape of 3 x 224 x 224 and a target mask shape of 1 x 224 x 224. Fixed image dimensions are necessary for batching and for the encoder-decoder architecture to downsample and upsample feature maps consistently.

Images are converted to tensors and normalized. The tensor conversion scales pixel values into the 0-1 range, and the image input is then normalized using mean values [0.485, 0.456, 0.406] and standard deviation values [0.229, 0.224,

0.225]. The masks are converted to tensors and kept in the 0-1 range, which is suitable for binary segmentation loss calculation.

2.3 Dataset Split

The dataset is divided into training, validation, and test sets using a two-step split. First, 70% of the data is assigned to training and 30% is kept as a temporary set. The temporary set is then divided equally into validation and test sets, resulting in the required 70/15/15 split.

Subset	Ratio	Purpose
Training	70%	Used to update model weights during learning.
Validation	15%	Used to monitor generalization and support early stopping.
Test	15%	Used after training for final model evaluation.

2.4 Data Augmentation

Data augmentation is applied only to the training set. This is the correct design because validation and test data should remain stable and representative of unseen data. The implemented augmentations include horizontal flipping, random cropping, brightness variation, and contrast variation.

Augmentation	Implementation purpose
Horizontal flip	Improves robustness to object location and left-right orientation.
Random crop	Exposes the model to slightly different object framing.
Brightness variation	Improves robustness to lighting differences.
Contrast variation	Improves robustness to visual intensity differences.

3. Architecture Design

The model is implemented as a custom encoder-decoder CNN. The encoder extracts increasingly abstract features from the input image, while the decoder reconstructs a pixel-level saliency mask. This design is suitable for segmentation tasks because the network must understand both the semantic object region and the spatial boundaries of that region.

3.1 Encoder

The encoder contains four convolutional blocks. Each block applies convolution, batch normalization, ReLU activation, a second convolution, another batch normalization, another ReLU activation, and then max pooling. Max pooling reduces the spatial resolution and increases the receptive field, allowing deeper layers to learn higher-level visual features.

Encoder block	Input channels	Output channels	Main operations
Encoder 1	3	32	Conv2D, BatchNorm, ReLU, Conv2D, BatchNorm, ReLU, MaxPool
Encoder 2	32	64	Conv2D, BatchNorm, ReLU, Conv2D, BatchNorm, ReLU, MaxPool
Encoder 3	64	128	Conv2D, BatchNorm, ReLU, Conv2D, BatchNorm, ReLU, MaxPool
Encoder 4	128	256	Conv2D, BatchNorm, ReLU, Conv2D, BatchNorm, ReLU, MaxPool

3.2 Bottleneck and Decoder

After the encoder, the model uses a bottleneck with 512 feature channels. The bottleneck contains convolutional layers, batch normalization, ReLU activation, and dropout. The dropout layer helps reduce overfitting by randomly disabling some activations during training.

The decoder contains four upsampling blocks. Each decoder block uses a ConvTranspose2D layer to increase spatial resolution and then concatenates the upsampled feature map with the corresponding encoder feature map through a skip connection. Skip connections are useful because they recover spatial details that may be lost during max pooling. This is especially important in SOD, where the predicted mask should align with object boundaries.

Decoder block	Input channels	Output channels	Main operations
Decoder 4	512	256	ConvTranspose2D, skip concatenation, Conv2D, BatchNorm, ReLU
Decoder 3	256	128	ConvTranspose2D, skip concatenation, Conv2D, BatchNorm, ReLU
Decoder 2	128	64	ConvTranspose2D, skip concatenation, Conv2D, BatchNorm, ReLU
Decoder 1	64	32	ConvTranspose2D, skip concatenation, Conv2D, BatchNorm, ReLU

3.3 Output Layer

The final layer is a 1 x 1 convolution that converts the decoder output into one channel. A sigmoid activation is applied to produce values between 0 and 1. The output mask therefore has the same spatial dimensions as the processed input image and can be compared directly with the ground-truth saliency mask.

4. Training Configuration

Training is implemented through a custom PyTorch training loop. The loop performs forward passes, loss calculation, backpropagation, optimizer updates, validation after each epoch, checkpoint saving, and early stopping. This satisfies the requirement to implement the training logic directly rather than relying on a high-level training wrapper.

4.1 Loss Function

The custom loss combines Binary Cross-Entropy with an IoU-based loss component:

$$\text{Total Loss} = \text{BCE}(\text{prediction}, \text{target}) + 0.5 \times (1 - \text{IoU})$$

Binary Cross-Entropy evaluates pixel-level correctness, while IoU loss encourages stronger region-level overlap between the predicted mask and the ground-truth mask. Combining both terms is appropriate for SOD because the model should not only classify individual pixels correctly but also produce a coherent object mask.

4.2 Optimizer, Epochs, and Early Stopping

The optimizer is Adam. The project requirement specifies a learning rate of 1e-3, and the training function supports lr=1e-3 as its default value. The final main execution in train.py uses a more conservative learning rate of 1e-4 and 30 epochs. This can be explained as an improvement run intended to make optimization more stable for the deeper encoder-decoder model.

The training script uses early stopping with a patience value of 10. If validation loss does not improve for 10 consecutive epochs, the training loop stops early. This helps prevent overfitting and avoids unnecessary computation after the model stops improving.

Configuration item	Value / behavior
Framework	PyTorch
Optimizer	Adam
Required learning rate	1e-3 supported by the train function default
Final script learning rate	1e-4 in main execution
Epoch range	Function supports 20 epochs by default; final script calls 30 epochs
Early stopping	Patience = 10 validation epochs without improvement

4.3 Checkpoint Saving and Resume Logic

The training loop saves the best-performing checkpoint whenever validation loss improves. The saved checkpoint includes the epoch number, model state dictionary, optimizer state dictionary, and validation loss. This is important because it preserves the best model version and stores enough information to continue training later.

The current implementation saves model and optimizer states, which are the required components for resume training. To fully automate resume behavior, the script would need to check for an existing checkpoint at startup, load the saved model weights, restore the optimizer state, restore the saved epoch number, and continue training from the next epoch. The checkpoint structure already supports this extension.

5. Evaluation Metrics

After training, the best checkpoint is loaded and evaluated on the test set. The evaluation script calculates IoU, Precision, Recall, F1-Score, Mean Absolute Error, and inference time. These metrics provide both region-overlap evaluation and pixel-level error analysis.

IoU measures overlap between the predicted mask and the target mask. Precision measures how many predicted salient pixels are correct. Recall measures how many true salient pixels were detected. F1-Score balances precision and recall. MAE measures the average absolute difference between predicted mask values and target mask values, where lower values are better.

Metric	Meaning	Direction
IoU	Intersection divided by union between predicted and ground-truth mask	Higher is better
Precision	Correct predicted salient pixels divided by all predicted salient pixels	Higher is better
Recall	Correct predicted salient pixels divided by all true salient pixels	Higher is better
F1-Score	Harmonic mean of precision and recall	Higher is better
MAE	Average absolute pixel difference between prediction and target	Lower is better

5.1 Test Results

The repository includes the evaluation functions, but the actual printed metric values must be inserted after running `python evaluate.py`. The final report should replace the placeholders below with the real output from the test evaluation.

Metric	Test result
IoU	0.5837
Precision	0.7040
Recall	0.8038
F1-Score	0.7121
MAE	0.1957
Average inference time	4.73 ms/image

6. Experiments and Improvements

The project includes several improvements over a minimal encoder-decoder CNN. These improvements are important because they show experimentation with both architecture and training configuration. The final model includes batch normalization, dropout, skip connections, deeper convolutional blocks, data augmentation, and a lower learning rate in the final training script.

Experiment	Description	Expected impact	Result
Baseline model	Basic encoder-decoder CNN with Conv2D, ReLU, MaxPooling, ConvTranspose2D, and Sigmoid output.	Creates the first working segmentation model.	Not separately recorded. Used as the initial model before improvements.
Batch normalization	BatchNorm layers added after convolutional layers in encoder, bottleneck, and decoder.	Stabilizes training and improves gradient flow.	Included in the final improved model.
Dropout in bottleneck	Dropout2d with probability 0.3 added in the bottleneck.	Reduces overfitting by regularizing deep feature activations.	Included in the final improved model.
Data augmentation	Horizontal flip, random crop, brightness variation, and contrast variation applied to training images.	Improves generalization to object position and lighting changes.	Included in the final improved model.
Learning-rate adjustment	Final script uses 1e-4 instead of the default 1e-3.	Makes training more stable for the improved architecture.	Final model result: IoU: 0.5837, Precision: 0.7040, Recall: 0.8038, F1-Score: 0.7121, MAE: 0.1957

The strongest architectural improvement is the use of skip connections. During encoding, max pooling reduces the spatial size of feature maps. While this helps the model learn abstract visual features, it can also remove boundary details. Skip connections transfer earlier encoder features into the decoder, helping the model recover sharper spatial information when reconstructing the output mask.

Batch normalization supports more stable learning because feature distributions are normalized inside the network. Dropout helps reduce overfitting, especially when the dataset is not very large. Data augmentation further improves generalization by exposing the model to variations in image position, crop, brightness, and contrast.

7. Visualizations

The evaluation script generates sample visualizations showing four views for each selected test image: the input image, the ground-truth saliency mask, the predicted saliency mask, and the overlay visualization. The output is saved to results/visualization.png.

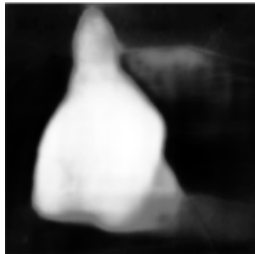
Input Image:



Ground-truth saliency mask:



Predicted saliency mask.



Overlay



Visual inspection is necessary because numerical metrics do not always reveal mask quality. A model may have acceptable IoU or F1-Score while still producing blurry edges, missing object parts, or false-positive background regions. The overlay view is especially useful because it shows whether the model is focusing on the correct object area in the original image.

8. Demonstration App

The project includes a Gradio demonstration app in app.py. The app loads the trained checkpoint, accepts a user-uploaded image, preprocesses it, runs the model in inference mode, and returns three outputs: the predicted saliency mask, an overlay visualization, and the inference time in milliseconds.

The user interaction is simple. First, the user uploads an image through the Gradio interface. The image is converted to RGB, resized to 224 x 224 pixels, converted to a tensor, and passed through the trained model. The output mask is converted into an image and resized back to the original input dimensions. Then the mask is overlaid on the red channel of the original image to make the salient region visible.

Step	Demo behavior
1	User uploads an image through the Gradio interface.
2	Image is converted to RGB and resized to 224 x 224 pixels.
3	Image tensor is passed through the trained SOD model.
4	The model predicts a one-channel saliency mask.
5	The predicted mask is resized to the original image size.
6	The app displays the mask, overlay, and inference time.

One recommended correction is to align the app preprocessing with the training preprocessing. The training data loader applies ImageNet-style normalization after `ToTensor()`, while the current app uses only resizing and `ToTensor()`. For more consistent inference, the same normalization used during training should also be applied in the demo app.

9. Environment and Tools

The project uses Python 3.9+ and PyTorch as the main deep learning framework. Supporting libraries are used for image processing, dataset splitting, visualization, progress tracking, and interactive inference.

Tool / Library	Purpose
Python 3.9+	Main programming language.
PyTorch	Model architecture, training, and inference.
torchvision	Image transformations and tensor preprocessing.
Pillow	Image loading and RGB/grayscale conversion.
NumPy	Array processing and metric aggregation.
scikit-learn	Train/validation/test split.
Matplotlib	Prediction and mask visualization.
tqdm	Progress bars during training and evaluation.
Gradio	Interactive demo for uploaded images.
Git / GitHub	Version control and project submission.

10. Conclusion

This project implements a complete Salient Object Detection pipeline using a custom CNN built from scratch. The system covers the full deep learning workflow: dataset loading, preprocessing, augmentation, model architecture, custom loss design, training, validation, checkpointing, evaluation, visualization, and demonstration through an interactive app.

The implementation satisfies the core technical requirements of the assignment. It uses a public SOD dataset, resizes image-mask pairs to a consistent input size, applies normalization and augmentation, implements an encoder-decoder CNN, produces a one-channel sigmoid saliency mask, trains with a custom BCE + IoU loss, evaluates with standard segmentation metrics, and provides qualitative visualizations.

Before final submission, the actual evaluation values from `evaluate.py` should be inserted into the results tables. It is also recommended to update `app.py` so the inference preprocessing matches the training preprocessing. With these final adjustments, the report and implementation provide a clear, professional-quality demonstration of an end-to-end machine learning and deep learning project.

References

- Project repository: <https://github.com/yllnuredini/sod-project>
- ECSSD dataset used through the local project folder data/ECSSD.
- PyTorch documentation and common segmentation training practices for CNN-based encoder-decoder models.